THE UNIVERSITY OF MELBOURNE

COMP30023 – Computer Systems

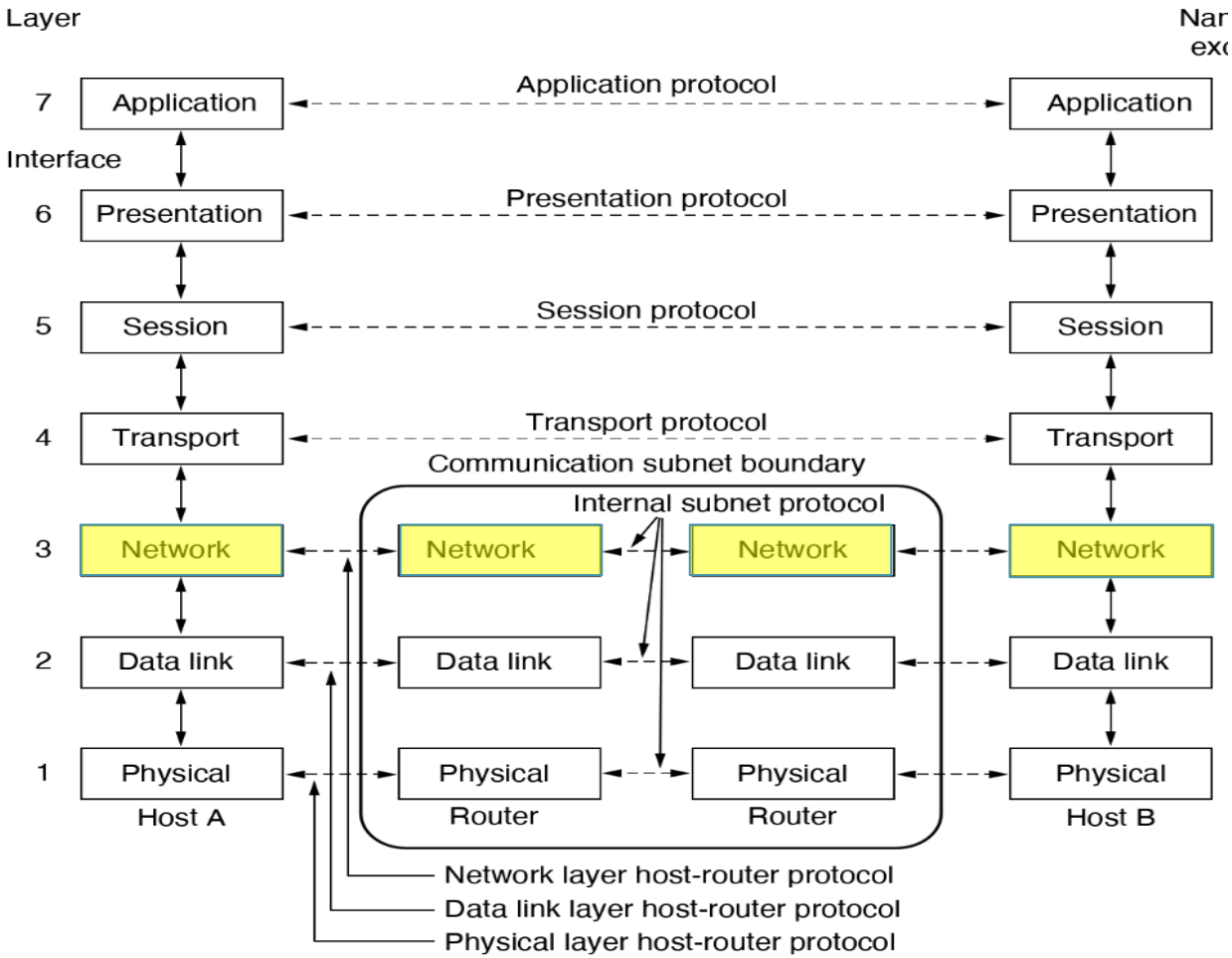
Internet (Network) Layer – Routing

Dr Lachlan Andrew

2025

© University of Melbourne 2022

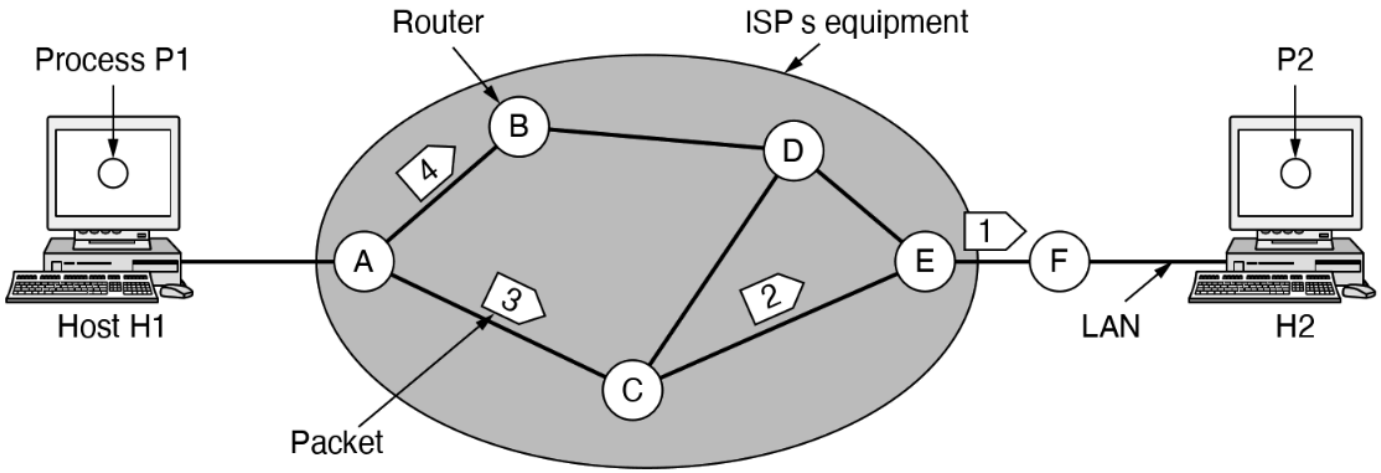
Position in the stack: Network layer



- Use data
- Tidy up e2e
- Get data e2e
- Tidy up p2p
- Get data p2p

© 2025 University of Melbourne

Recap: Packet forwarding



A s table (initially)

Dest.	Line
A	–
B	B
C	C
D	B
E	C
F	C

A s table (later)

A	–
B	B
C	C
D	B
E	B
F	B

C s table

A	A
B	A
C	–
D	E
E	E
F	E

E s table

A	C
B	D
C	C
D	D
E	–
F	F

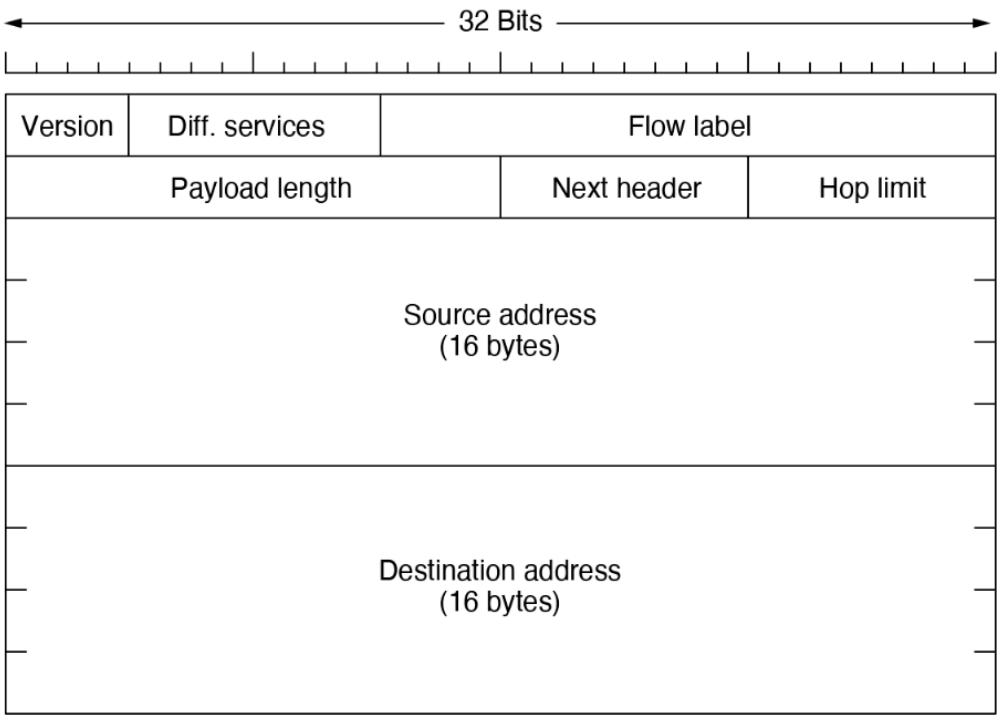
TN 6th 5-2

IPv6

- Designed over 20 years ago to address the problem of exhausting the IPv4 address space
- While solving that problem, some other changes were made
  - Simpler header – allows faster processing
  - Improved security – now back-ported to IPv4
  - Further Quality of Service support
- IPv6 addresses are 128 bits
  - Unlikely ever to run out
- Check out if you can use IPv6: http://test-ipv6.com

IPv6 Header

Field	Usage
Version	6
Differentiated services	6 bits for service class, 2 bits for congestion control (ECN)
Flow label	Pseudo-Virtual Circuit identifier
Payload length	Bytes after the 40 byte header
Next header	Used to specify additional headers or Protocol (TCP/UDP)
Hop limit	Same as TTL (Time To Live)
Source and Destination	16 byte IPv6 addresses



TN 6th 5-57

IPv6 Addressing

- Written as 8 groups of (up to) 4 hex digits
  - 8000:0000:0000:0000:0123:4567:89AB:CDEF
  - [8000:0000:0000:0000:0123:4567:89AB:CDEF]:port_number
- Can be optimised by stripping *one* group of consecutive 0's
  - 8000::123:4567:89AB:CDEF
- Backwards compatibility with IPv4 is achieved with
  - ::ffff:192.31.2.46 (note the mix of hex with decimal)
- Still not widely deployed
  - Google measures IPv6-*capable* requesters, https://www.google.com/intl/en/ipv6/statistics.html
  - still around 40% globally, (~32% in Australia)
- Fairly widely supported – likely to see a continued growth

IPv6 addresses are larger than IPv4

```
    // #define required for VSCode
#define _POSIX_C_SOURCE 200112L
#include <netdb.h>
#include <string.h>
#include <stdio.h>
#include <unistd.h>

int connfd = 0, re = 1, s, n;
struct addrinfo hints, *res, *rp;

memset(&hints, 0, sizeof hints);
hints.ai_family = AF_INET6;
hints.ai_socktype = SOCK_STREAM;
hints.ai_flags = AI_ACTVE;
```

```
s = getaddrinfo("localhost",
                "5000", &hints, &res);
for (rp = res; rp != NULL;
            rp = rp->ai_next) {
    connfd = socket(rp->ai_family,
                    rp->ai_socktype,
                    rp->ai_protocol);
    if (connfd == -1) continue;
    if (connect(connfd,
                rp->ai_addr,
                rp->ai_addrlen) != -1
        break;
    close(connfd);
}
write (connfd, "Hello, network!\r\n");
close (connfd);
```

Some, like phone, have variable sizes

© 2025 University of Melbourne

Summary

- Forwarding vs Routing
- Routing within networks / domains

© 2025 University of Melbourne

Packet forwarding

- Each router has a forwarding table (or routing table).
- This maps destination addresses to outgoing interfaces.
- Upon receiving a packet:
  - inspect the destination IP address in the header
  - index into the table
  - determine the outgoing interface
  - forward the packet out that interface
- Then, the next router in the path repeats the process
- The packet travels along the path to the destination.

© 2025 University of Melbourne

Routing

- How do these forwarding tables get *created*?

- The *routing algorithm* decides which output line an incoming packet should be transmitted on
- Combination of
  - an algorithm local to each router
  - a protocol to gather the network information needed by the algorithm

© 2025 University of Melbourne

Routing Algorithm

- Properties of a good routing algorithm
  - Correctness – finds a valid route between all pairs of nodes
  - Simplicity
  - Robustness – a router crash should not require a ‘network’ reboot
  - Stability – a stable algorithm reaches equilibrium and stays there
  - Fairness
  - Efficiency
  - Flexibility to implement policies

© 2025 University of Melbourne

Delay vs. Bandwidth

- What is being optimised?
  - Mean packet delay?
  - Max network throughput?

- Simplest approach
  - Minimise the number of hops a packet has to make
    - Tends to reduce per packet bandwidth and improve delay
    - Hopefully also reduces the distance travelled – but not guaranteed
      - Crossing the Pacific is one IP hop
  - Actual algorithms give a cost to each link
    - More flexible, but still cannot express all routing preferences

© 2025 University of Melbourne

Routing Algorithm

- Non-adaptive (static routing)
  - Does not adapt to the network topology
  - Calculated ‘offline’ and uploaded to the router at boot
  - Does not respond to failure
  - Reasonable where there is a clear or implicit choice
    - Think about your home router, a static route out of your network is perfectly reasonable, since it is the only choice
- Adaptive
  - Dynamic routing, adapts to changes in topology and potentially even traffic levels
  - Optimise some property: distance, hops, estimated transit time, etc.
  - May get information from adjacent routers, or all routers in the network

© 2025 University of Melbourne

Flooding

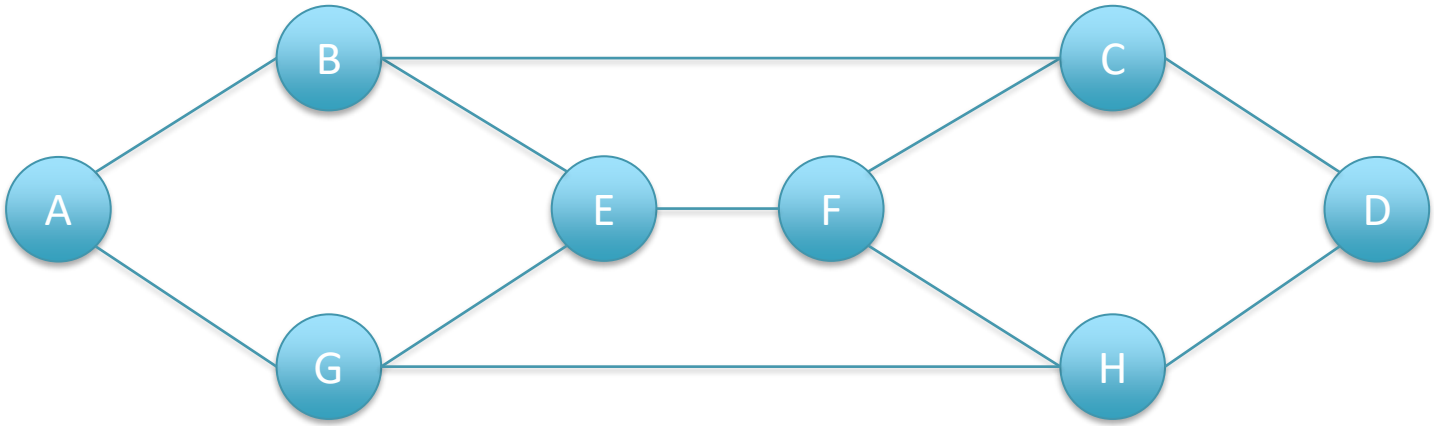
- Send packet to all neighbours who don't have it yet

- Simplest non-static routing
  - Guarantees shortest distance and minimal delay
  - Useful benchmark in terms of speed
  - Extremely robust – if there is a path it will find it
  - Highly inefficient – generates many duplicate packets
  - Have to have a way of discarding packets (TTL)
    - If unknown can be set to diameter of network

© 2025 University of Melbourne

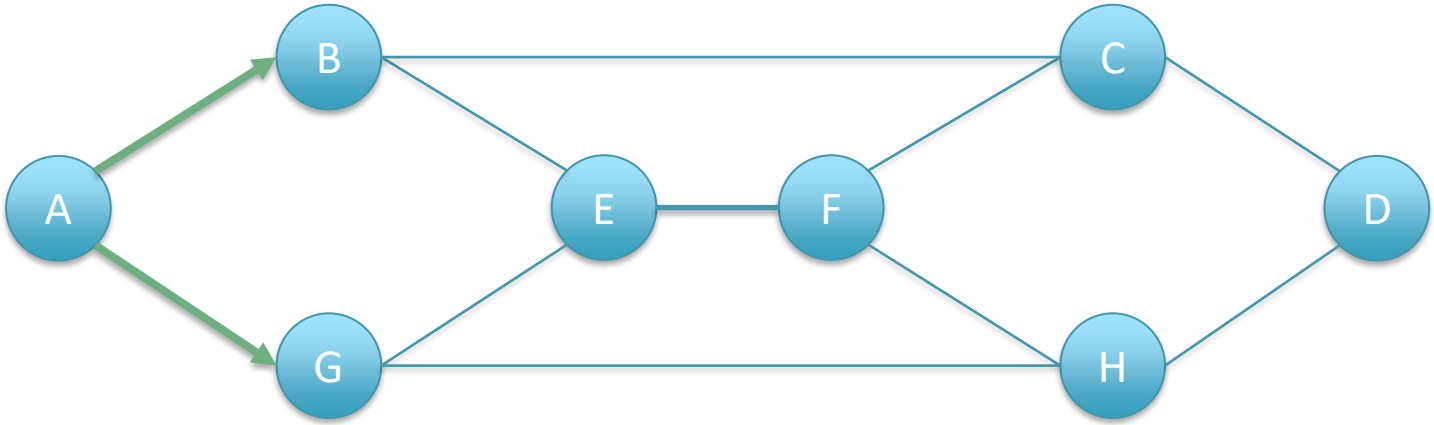
Flooding

- Send a packet from A to D



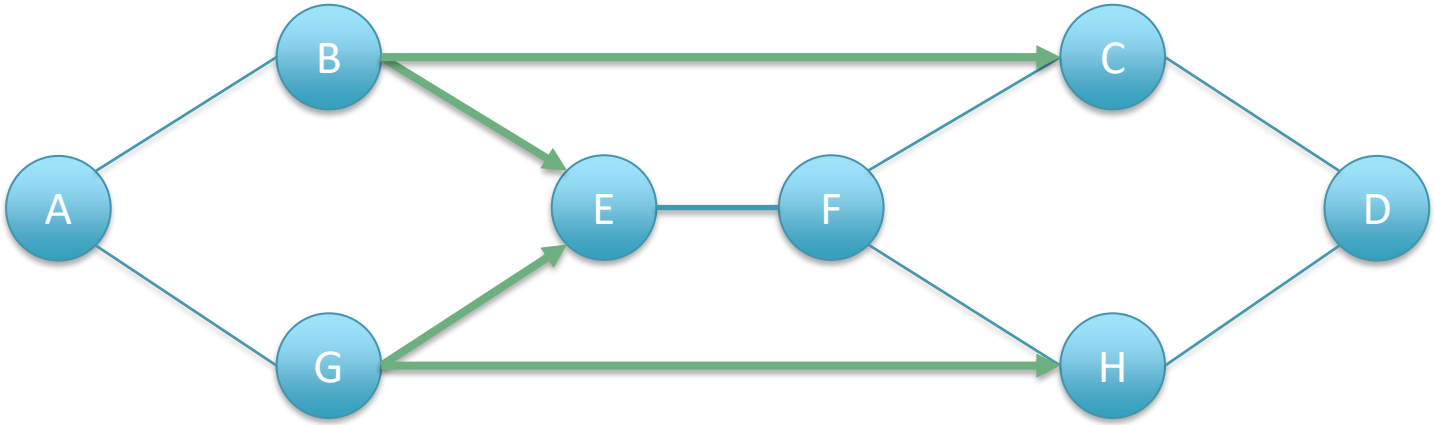
© 2025 University of Melbourne

Flooding



© 2025 University of Melbourne

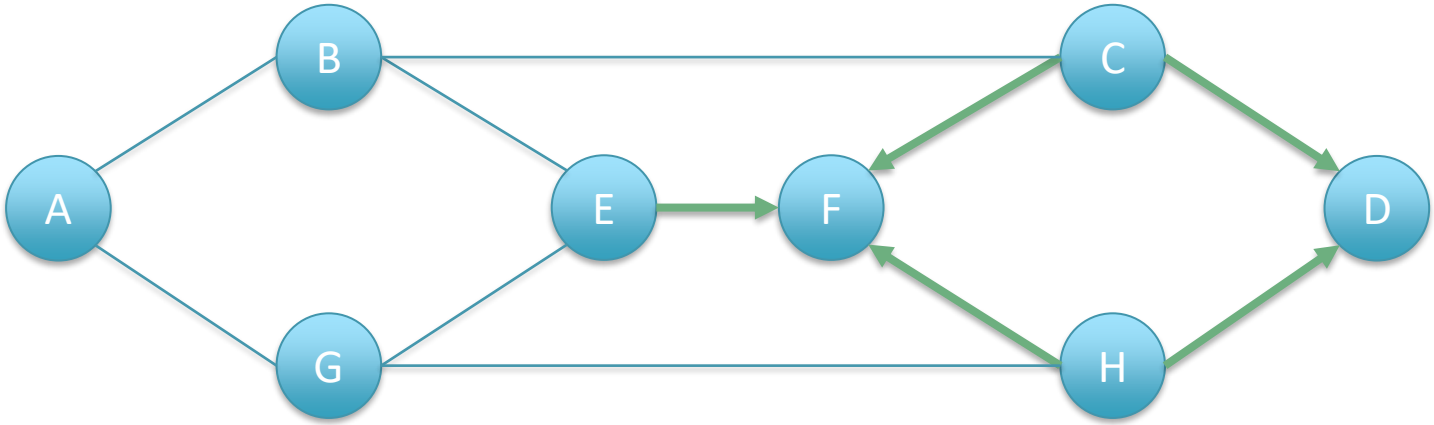
Flooding



E receives two copies

© 2025 University of Melbourne

Flooding



E forwards only one copy
Must keep track of packets it has forwarded

© 2025 University of Melbourne

Adaptive Routing

- We need something more efficient than flooding
- Should adapt to network topology and changes
- If we have complete knowledge of the network “topology” (what is connected to what) how can we determine an optimal route?

© 2025 University of Melbourne

Optimality Principle (Bellman 1957)

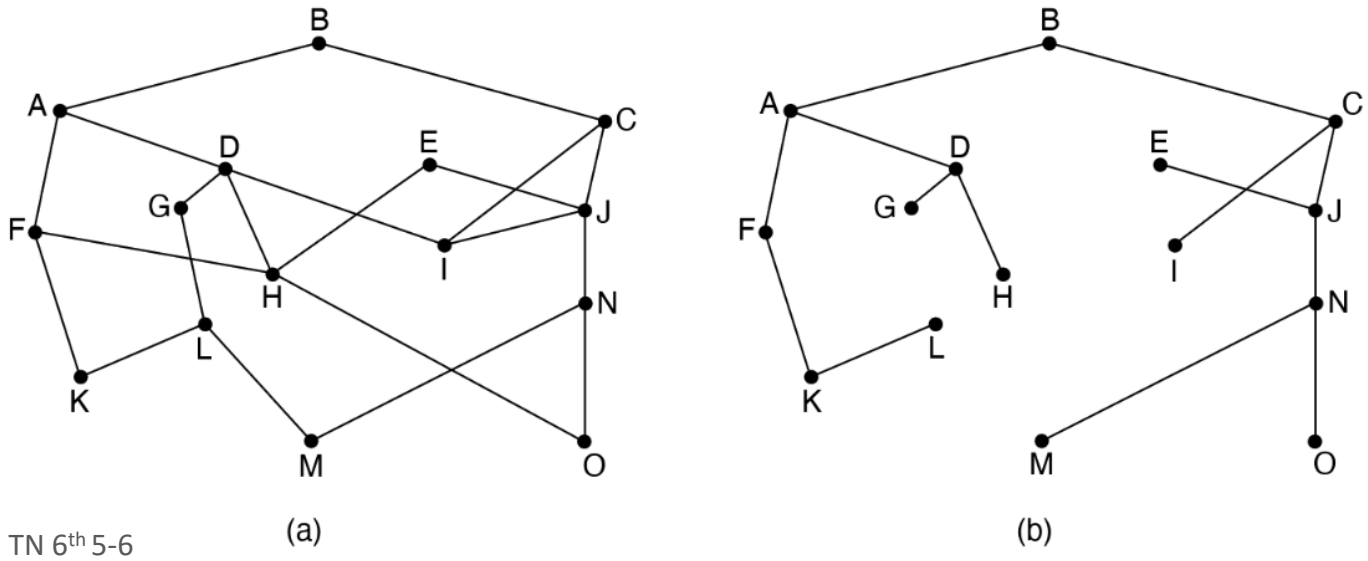
- If router *J* is on the optimal path from router *I* to *K*, then the optimal path from *J* to *K* also falls along the same route.
  - If a better route existed for *J* to *K* it would be combined with the one from *I* to *J*, which would contradict our initial condition that our route from *I* to *K* was optimal

- When we study BGP, we'll see that this doesn't always apply

© 2025 University of Melbourne

Sink Tree

- The optimality principle means that a set of optimal routes from all sources form a tree rooted at the destination



TN 6th 5-6

network

sink tree for router B

© 2025 University of Melbourne

Shortest Path Algorithm

- View as a labelled graph
  - Label weight based on delay, distance, cost, etc.
- A number of algorithms, most famous is Dijkstra's algorithm
  - Finds the shortest path between a sink and all sources or vice versa

- At each step, each node is labelled with its distance (sum of costs on edges) from the source, and the best known path
  - Distances must be non-negative

© 2025 University of Melbourne

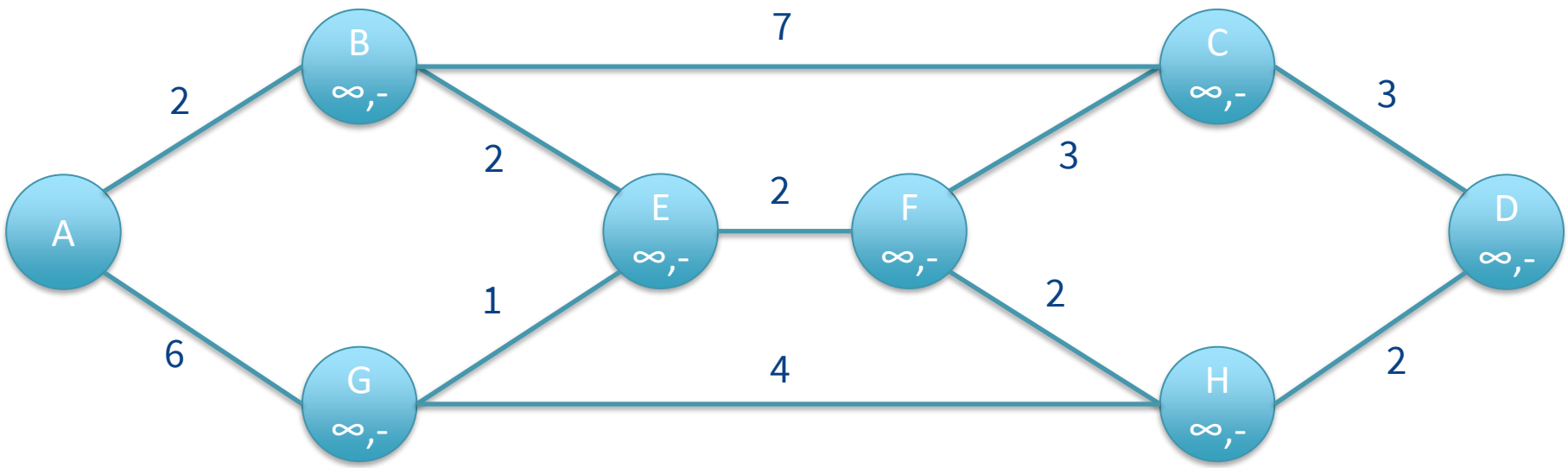
Dijkstra's Algorithm

- Divide nodes into three groups: "unseen", "open", "closed"
  - unseen: Not a neighbour of any node we have processed
  - Open: We have "visited" a neighbour, but not it. We know *a* path
  - Closed: We have visited it. We know the *best* path to it
- The algorithm moves nodes: unseen -> open -> closed
- Initially no paths are known so all nodes have a value of infinity ("unseen")
- As the algorithm proceeds, labels will be updated
  - Tentative ("open") – initial state
  - Permanent ("closed") – once a shortest path is found label is permanent and won't change again

© 2025 University of Melbourne

Shortest Path

- Send a packet from A to D



Dijkstra's Algorithm

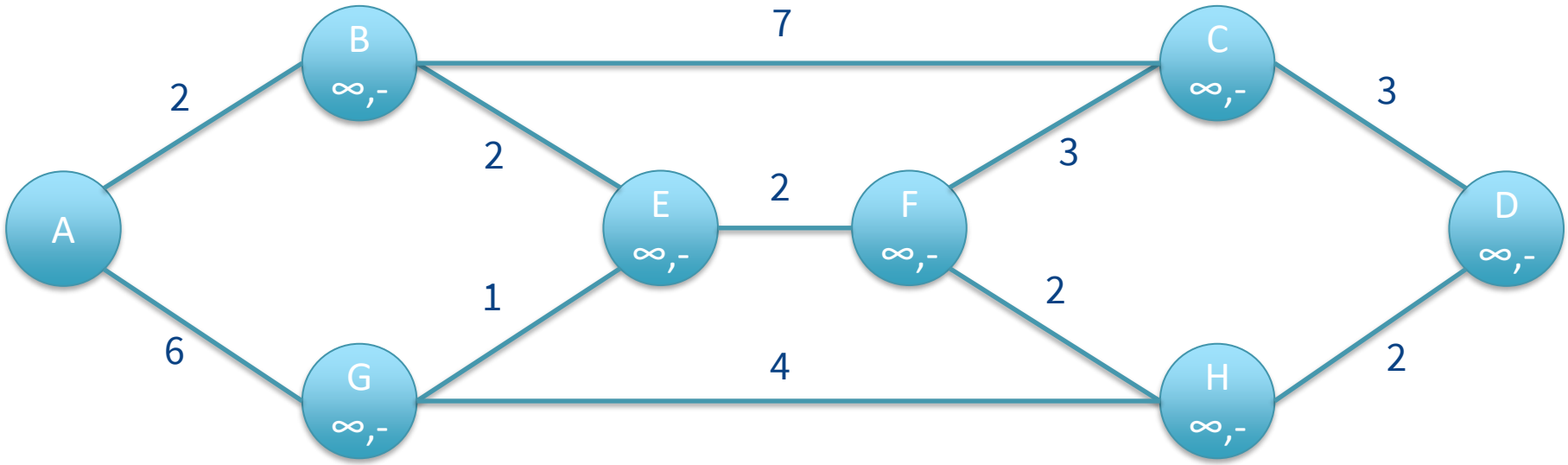
- Visit the source node: "Open" all of its neighbours and set their labels as the distance to them.
- Repeat until all nodes visited, or destination found:

1. Examine adjacent nodes to the "*working node*", calculate distance, update labels if improved
2. Examine all tentative/open nodes in the graph and pick the one with the lowest distance and "visit" it
   - make that permanent/closed,
   - mark it as the "working node"
   - Go to step 1. (i.e., "Open" all of its neighbours and calculate the distance to them via this path. If it is *less* than the neighbour's current label set the label to the lower value.)

© 2025 University of Melbourne

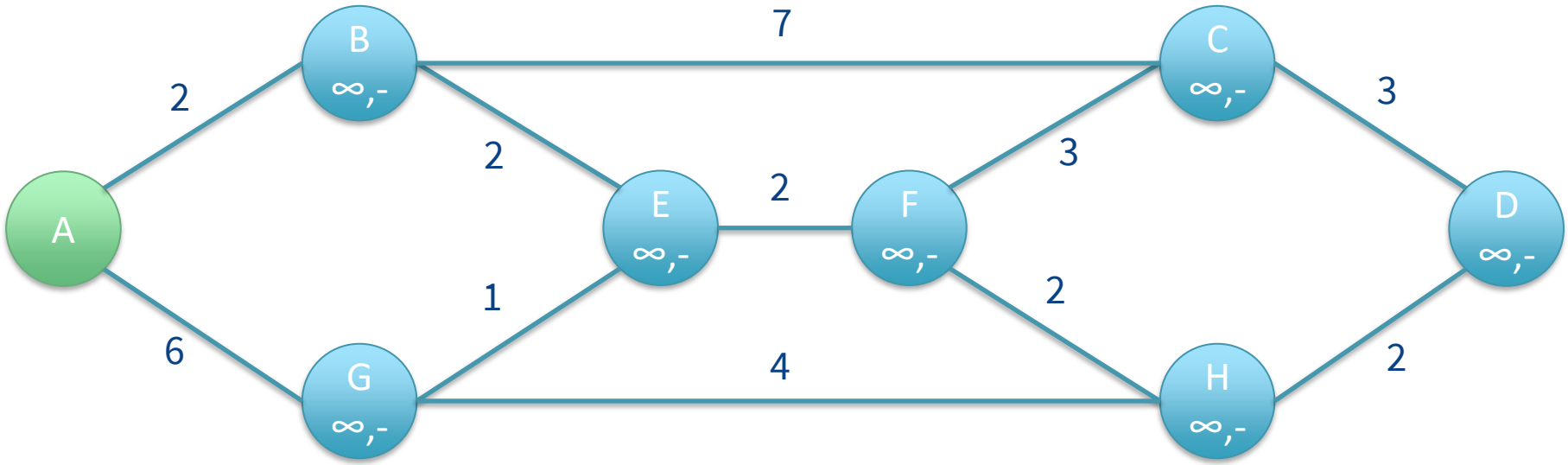
Shortest Path

- Send a packet from A to D



Shortest Path

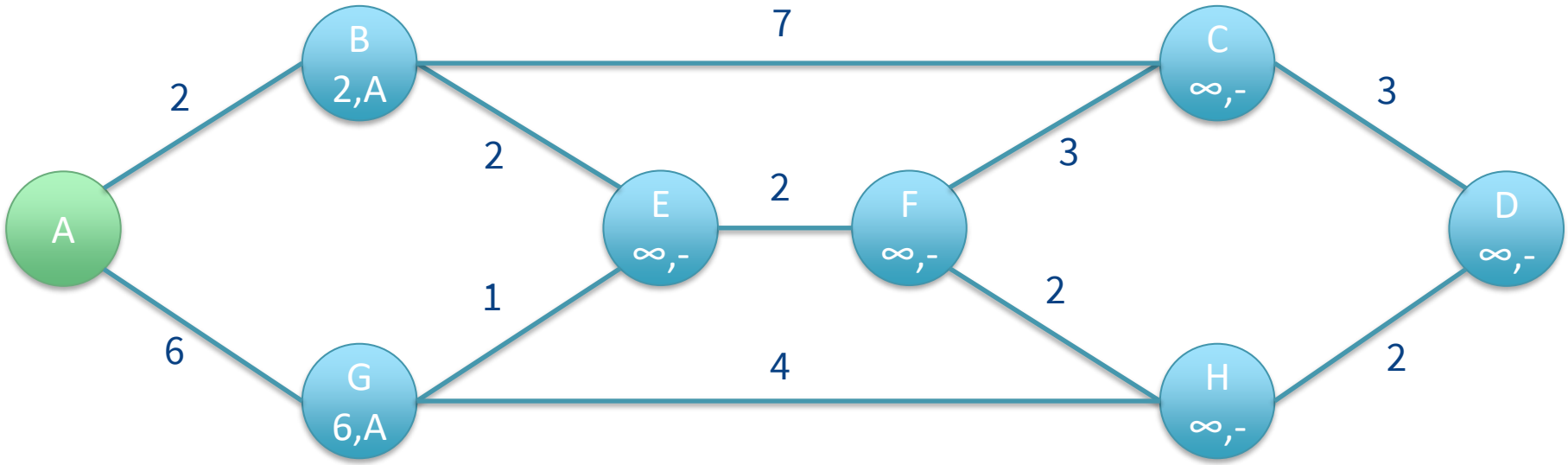
Make A permanent



© 2025 University of Melbourne

Shortest Path

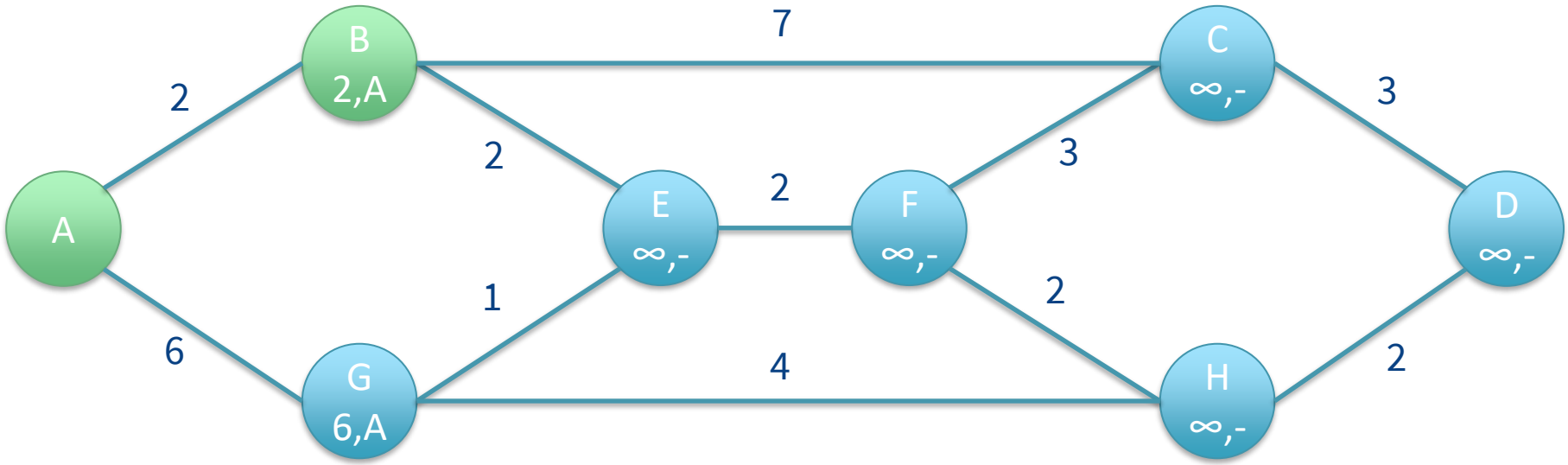
Open adjacent nodes (B,G): recalculate their weights



© 2025 University of Melbourne

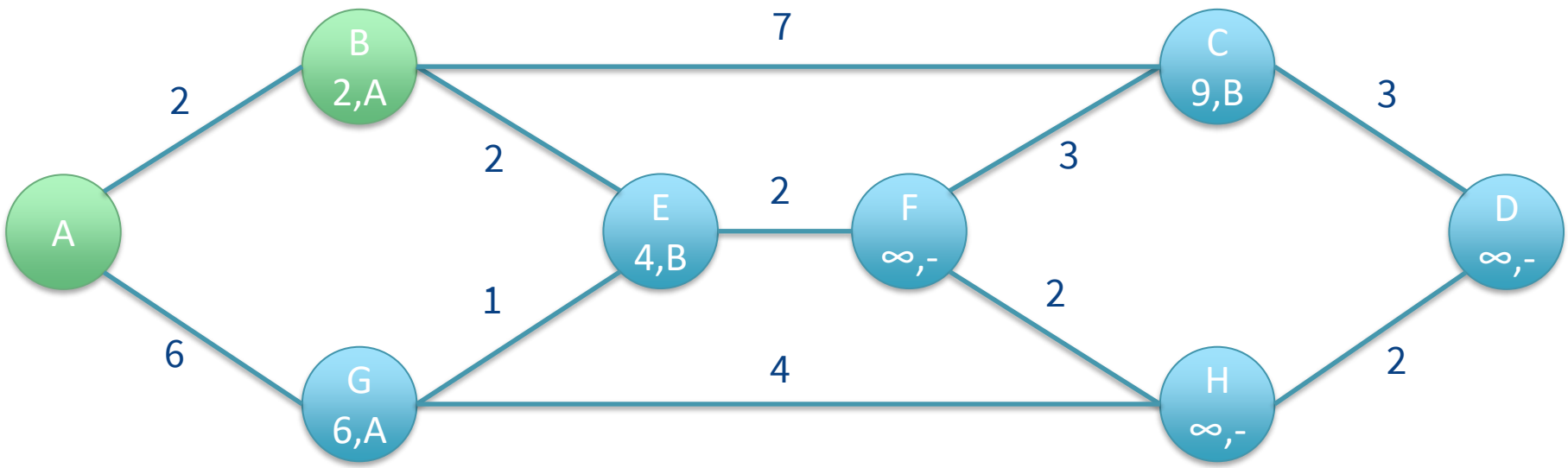
Shortest Path

Select lowest distance tentative node (B) and make permanent



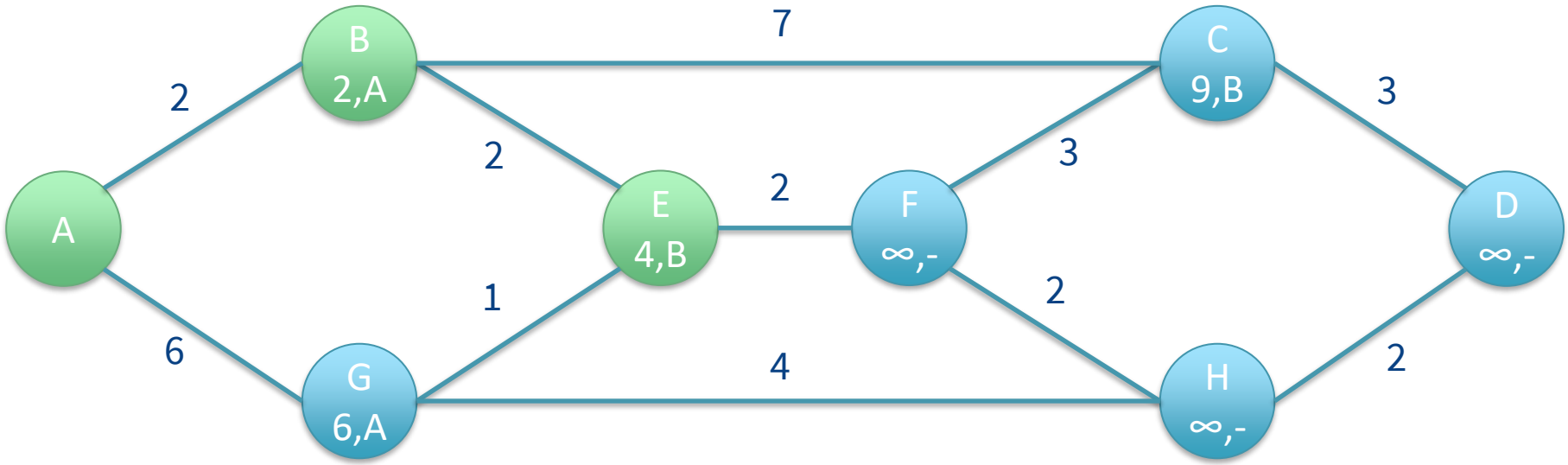
Shortest Path

- Recalculate distance for adjacent nodes (C,E)



Shortest Path

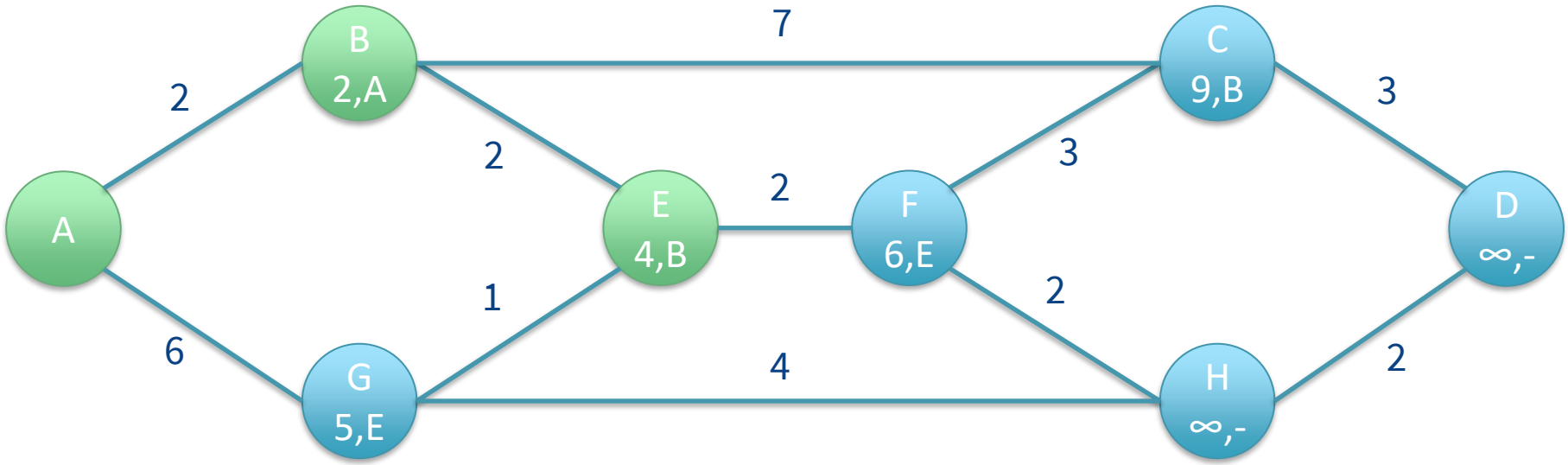
Select lowest distance tentative node (E) and make permanent



© 2025 University of Melbourne

Shortest Path

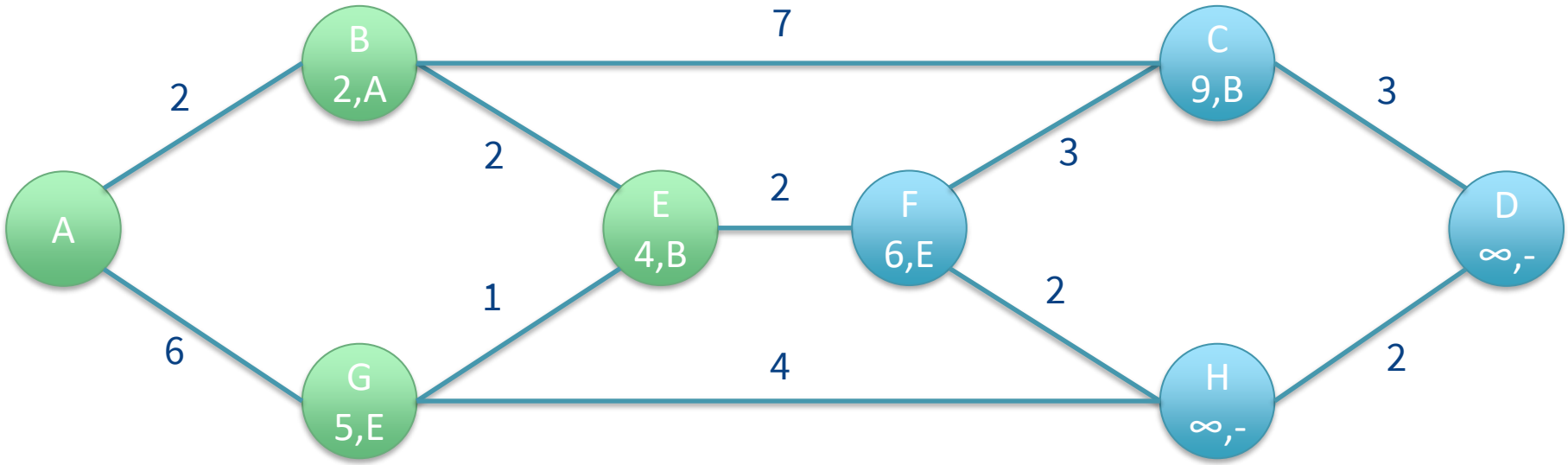
Recalculate distance for adjacent nodes (F, G). Lower G's cost



© 2025 University of Melbourne

Shortest Path

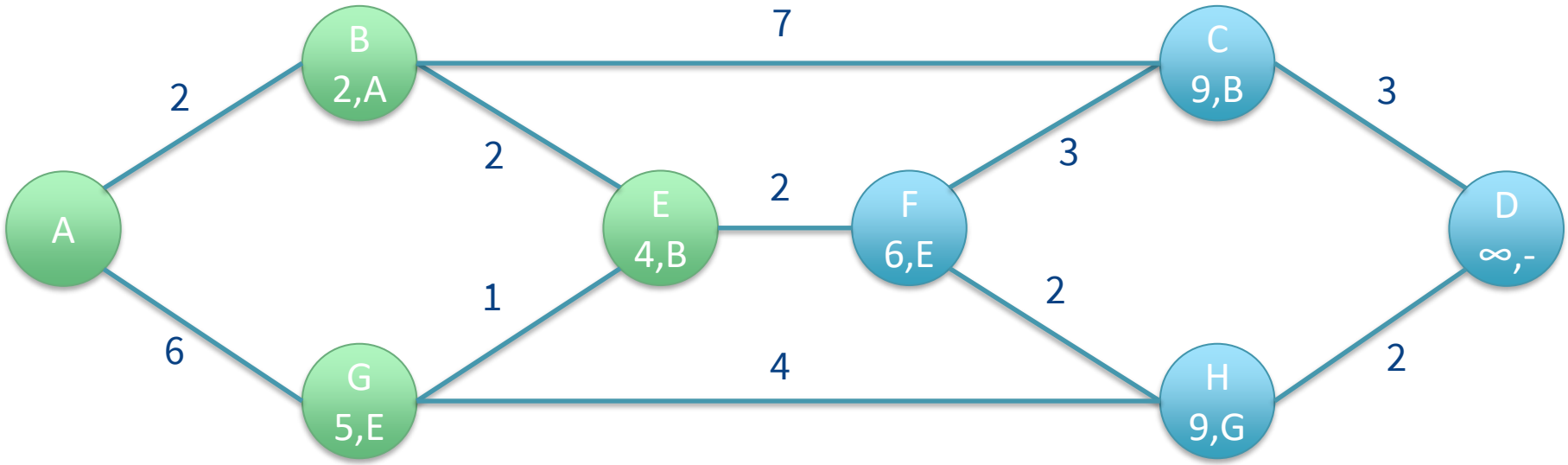
Select lowest distance tentative node (G) and make permanent



© 2025 University of Melbourne

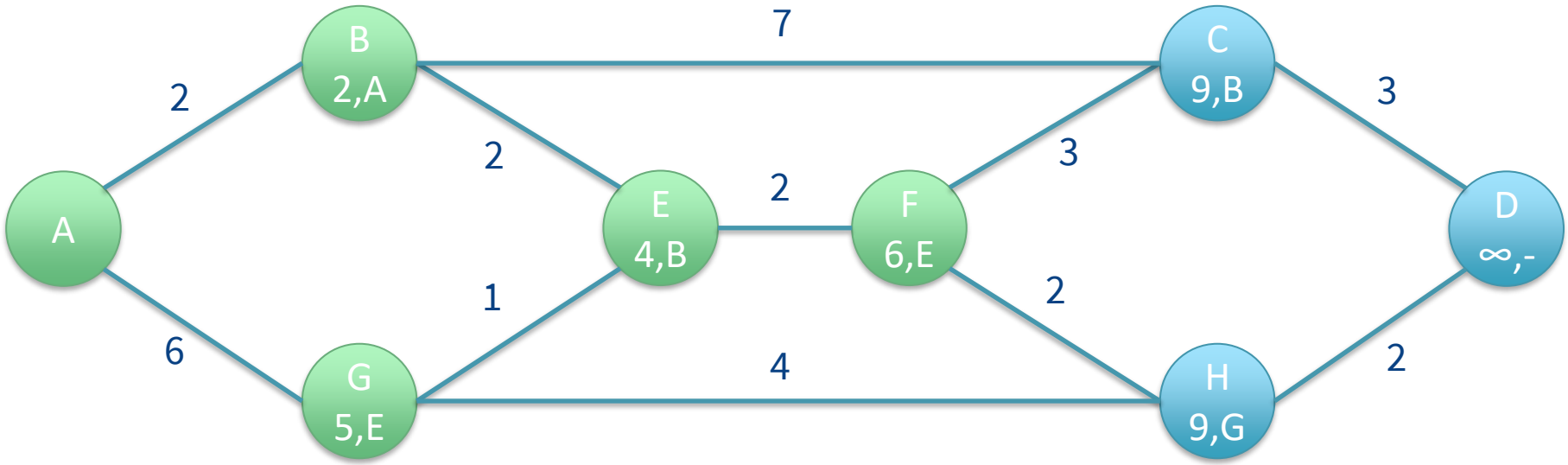
Shortest Path

Recalculate distance for adjacent nodes (H)



Shortest Path

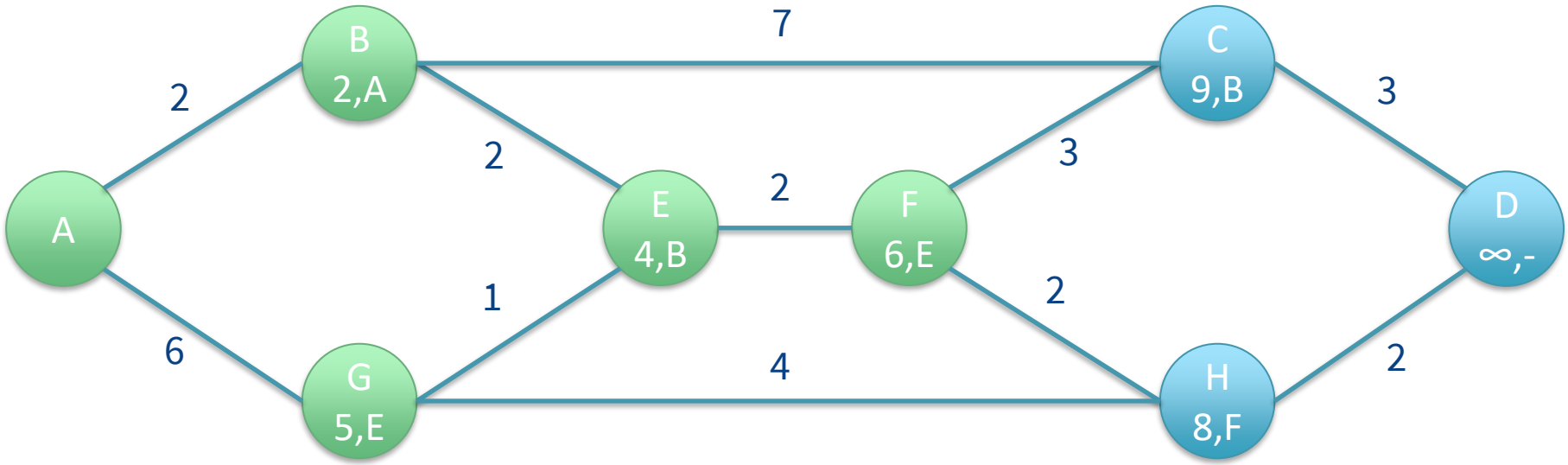
Select lowest distance tentative node (F) and make permanent



© 2025 University of Melbourne

Shortest Path

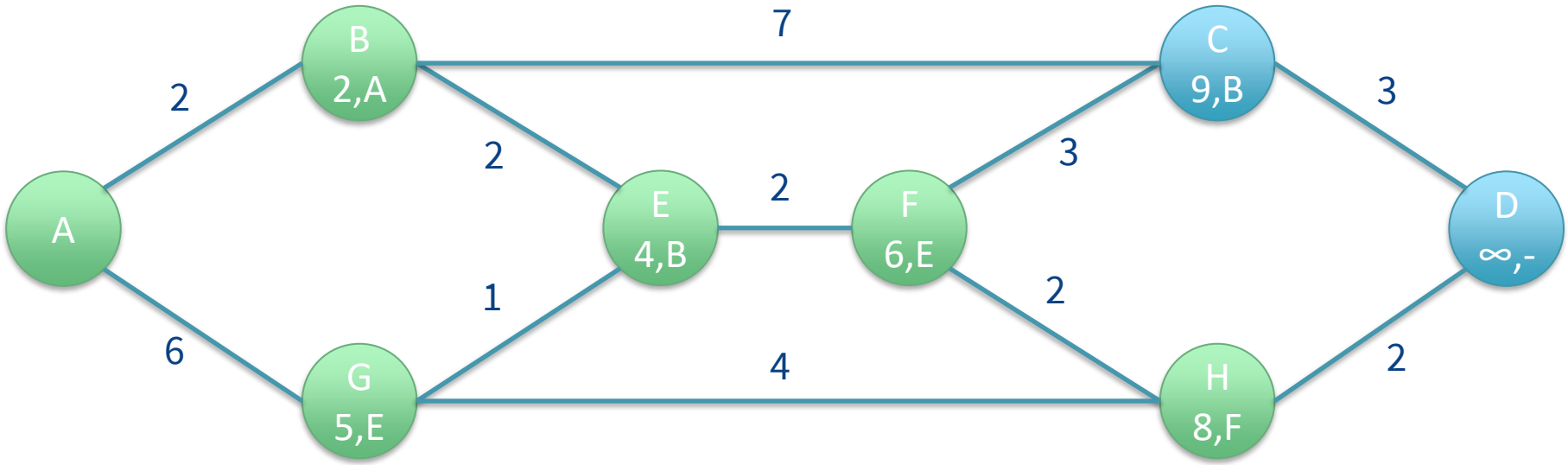
Recalculate distance for adjacent nodes (H drops, C stays)



© 2025 University of Melbourne

Shortest Path

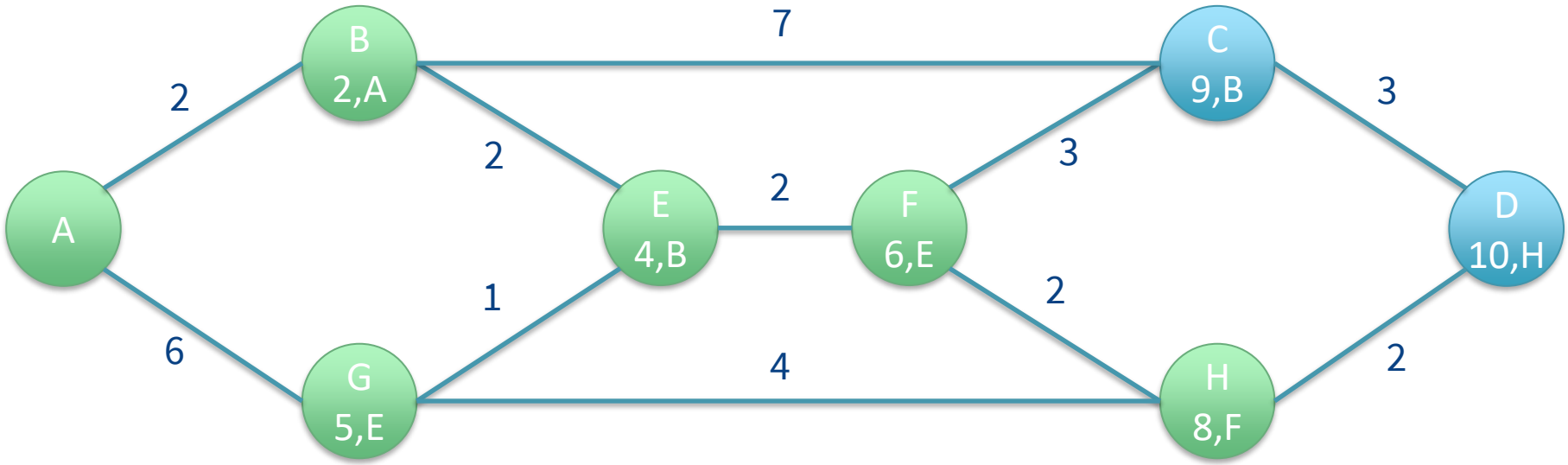
Select lowest distance tentative node (H) and make permanent



© 2025 University of Melbourne

Shortest Path

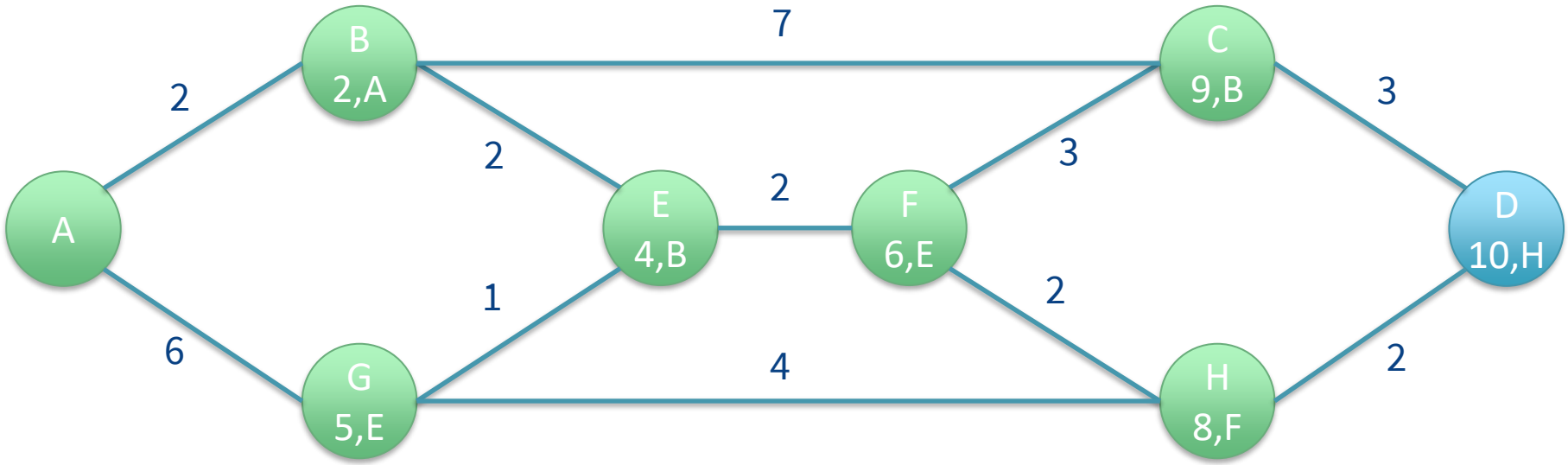
Recalculate distance for adjacent nodes (D)



© 2025 University of Melbourne

Shortest Path

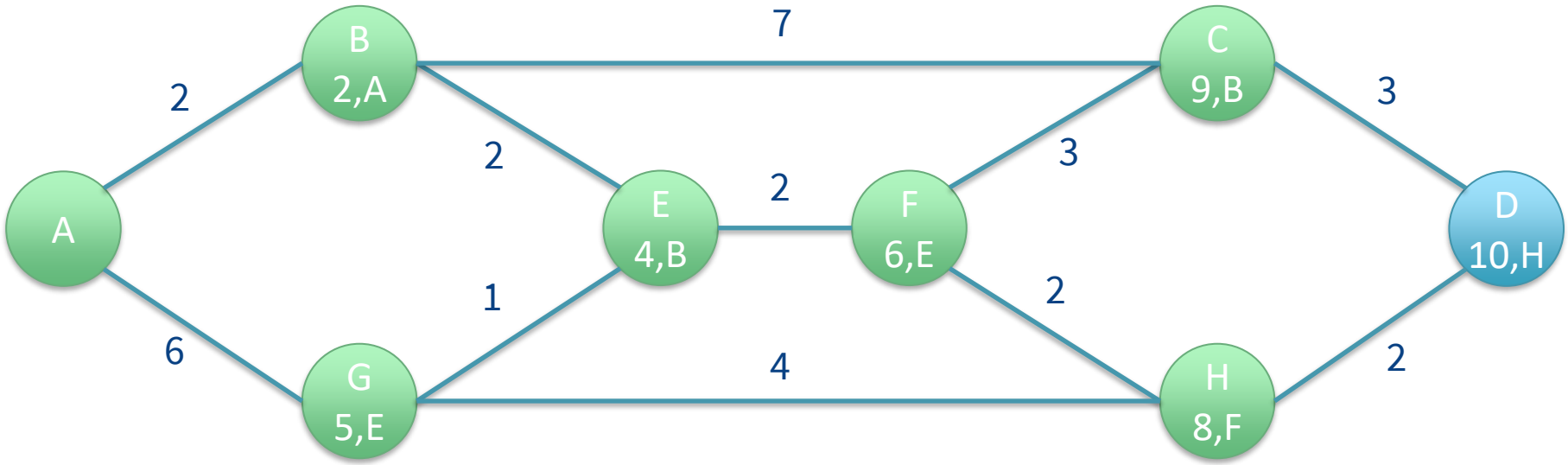
Select lowest distance tentative node (C) and make permanent



© 2025 University of Melbourne

Shortest Path

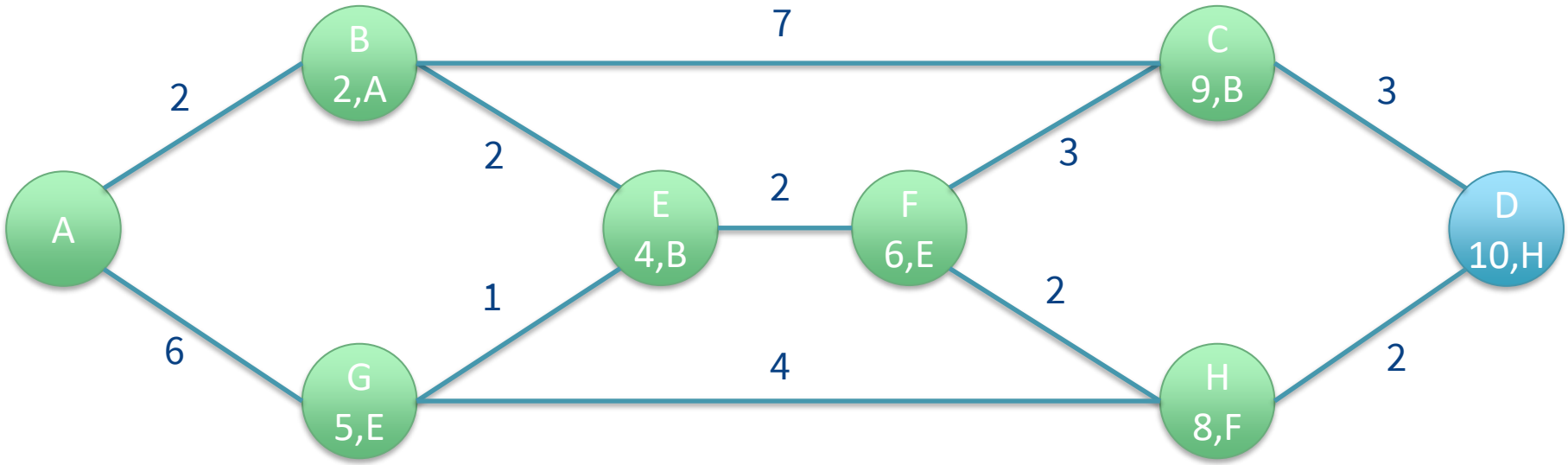
Recalculate distance for adjacent nodes (D, stays)



© 2025 University of Melbourne

Shortest Path

We can now stop because the lowest cost tentative node is the destination



© 2025 University of Melbourne

Link State Routing

- LS is a “distributed” algorithm that replaced Distance Vector Routing (“Bellman-Ford”), which converged slowly
- Variants of Link State Routing used today
- Simple 5 step process at each router:

Share info

1. Discover its neighbours and learn their network address
2. Set the distance or cost metric to each of its neighbours
3. Construct a packet containing all it has just learned
4. Send the packet to, and receive packets from, all other routers

Centralized alg

5. Compute the shortest path to every other router

© 2025 University of Melbourne

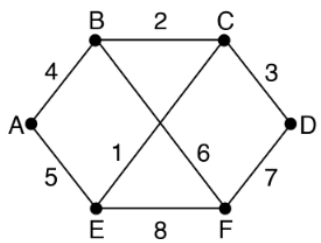
Link State Routing

- To discover neighbours, a router on boot sends out a HELLO packet on each interface. The router on the other end must reply with its unique ID
  - Slightly more complicated on LANs
- Cost can be set automatically or manually
  - Common technique is 1/bandwidth: 1 Gbps = 1, 100 Mbps = 10
  - Could use delay as well – calculated using an ECHO packet
  - Many networks manually choose preferred routes and then look for link costs to make those routes the shortest. "Traffic engineering"

© 2025 University of Melbourne

Link State Routing

- A Link State packet consists of ID, sequence number, age, and a list of neighbours and their respective costs



(a)

Link State Packets

A	
Seq.	
Age	
B	4
E	5

B	
Seq.	
Age	
A	4
C	2
F	6

C	
Seq.	
Age	
B	2
D	3
E	1

D	
Seq.	
Age	
C	3
F	7

E	
Seq.	
Age	
A	5
C	1
F	8

F	
Seq.	
Age	
B	6
D	7
E	8

(b)

TN 6th 5-12

- Building the packet is easy, deciding *when* to build them is difficult
  - At intervals?
  - When a change occurs – link disconnect?

© 2025 University of Melbourne

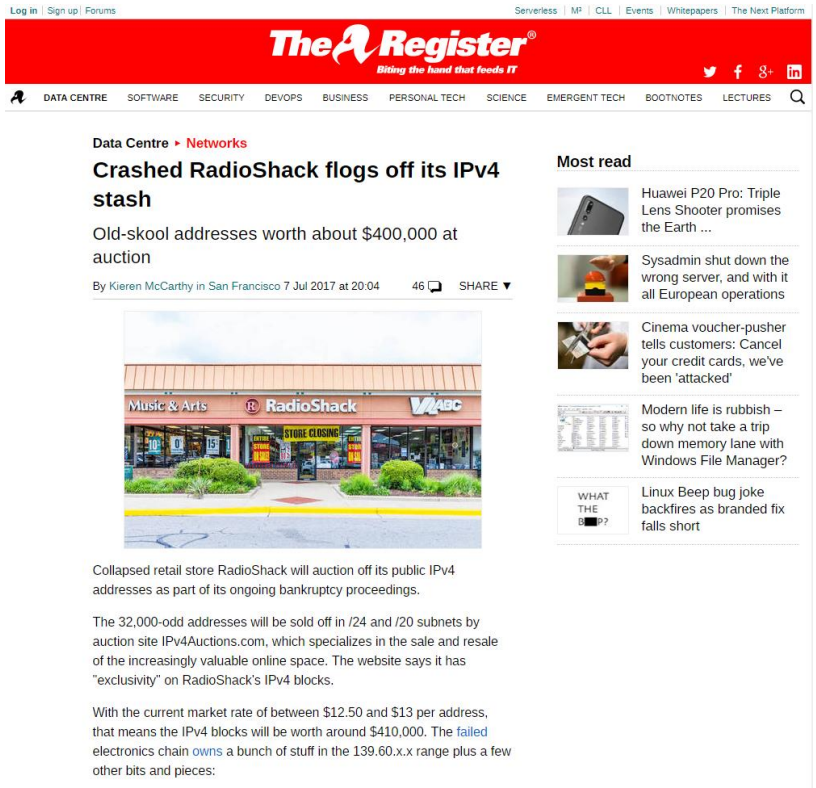
Link State Routing

- To send packets to all other routers, flooding is used
  - To be precise, reliable flooding – which uses acknowledgements to guarantee every other router receives the packet
  - To avoid waste, when a router receives a Link State Packet it compares the sequence number to the one it previously received (if any). If the sequence number is not larger it discards it and does not forward on the flood
  - Sequence numbers are 32 bits to avoid wrap-around
  - Still a problem if a router crashes and restarts its sequence number from 0
    - Solution is the age field, which is reduced by 1 each second, when the age hits zero the information is discarded

© 2025 University of Melbourne

And finally…

- One of the disadvantages of a hierarchical address space is that it can be quite wasteful if not carefully assigned, i.e. large parts of the address space unused if blocks badly assigned
- Combined with the scarcity (exhaustion) of available IP addresses, they have become a valuable commodity
- Not intended to be sold, should really be returned to the assigning body for reallocation
- Early adopters of IPv6 were able to sell their IPv4 address space at a premium



Log in | Sign up | Forums

Serverless | MP | CLL | Events | Whitepapers | The Next Platform

The Register®
Biting the hand that feeds IT

DATA CENTRE SOFTWARE SECURITY DEVOPS BUSINESS PERSONAL TECH SCIENCE EMERGENT TECH BOOTNOTES LECTURES

Data Centre ▸ Networks

Crashed RadioShack flogs off its IPv4 stash

Old-skool addresses worth about $400,000 at auction

By Kieren McCarthy in San Francisco 7 Jul 2017 at 20:04 46 SHARE ▼

Collapsed retail store RadioShack will auction off its public IPv4 addresses as part of its ongoing bankruptcy proceedings.

The 32,000-odd addresses will be sold off in /24 and /20 subnets by auction site IPv4Auctions.com, which specializes in the sale and resale of the increasingly valuable online space. The website says it has "exclusivity" on RadioShack's IPv4 blocks.

With the current market rate of between $12.50 and $13 per address, that means the IPv4 blocks will be worth around $410,000. The failed electronics chain owns a bunch of stuff in the 139.60.x.x range plus a few other bits and pieces:

Most read

Huawei P20 Pro: Triple Lens Shooter promises the Earth ...

Sysadmin shut down the wrong server, and with it all European operations

Cinema voucher-pusher tells customers: Cancel your credit cards, we've been 'attacked'

Modern life is rubbish – so why not take a trip down memory lane with Windows File Manager?

Linux Beep bug joke backfires as branded fix falls short

Acknowledgement

- The slides were Adapted by Lachlan Andrew from those prepared by Chris Culnane based on material developed previously by: Michael Kirley, Zoltan Somogyi, Rao Kotagiri, James Bailey and Chris Leckie.
- Some of the images included in the notes were supplied as part of the teaching resources accompanying the text books listed in lecture 1.
  - (And also) Computer Networks, 6th Edition, Tanenbaum A., Wetherall. D. https://ebookcentral.proquest.com/lib/unimelb/detail.action?docID=6481879
- Textbook Reference: Ch 5.1-5.4

© University of Melbourne